# Стохастический градиентный спуск. Часть 1

На занятии по обучению нейронных сетей при работе с моделью, предсказывающей рукописные цифры, мы столкнулись с двумя сложностями:

1. из-за большого количества наблюдений модель медленно обучалась; и кроме того
2. уровень ошибки при заданном количестве эпох и скорости обучения снизился недостаточно, что привело к слабым результатам как на обучающей, так и на тестовой выборках.

На сегодняшнем занятии мы рассмотрим способы улучшения метода градиентного спуска. Мы уже начали этот разговор на последнем занятии вводного курса, отвечая на вопросы про параметр batch-size.

## Терминология и нотация

Вначале договоримся о терминах и нотации. Функция потерь или **целевая функция** с одним или несколькими параметрами $\theta$ будет обозначаться как $f(\theta)$. Точку **минимума** такой функции, то есть оптимальные веса, обозначим $\theta^*$. **Градиент** функции в точке $\theta$ — $\nabla f(\theta)$.

**Данные** поместим в переменные $(X, y)$ для всего набора данных, и $(x_i, y_i)$ — для отдельных **наблюдений**. Дополнительно заметим, что в литературе по оптимизации параметры часто обозначают через $x$, а целевую функцию соответственно $f(x)$. На данном этапе будем избегать такой нотации, чтобы не путать параметры и наблюдения.

**Коэффициент скорости обучения** (learning rate) обозначим буквой $\eta$ («эта»); также заметим, что в литературе по оптимизации $\eta$ называют **размером шага** (step size).

Итерации и эпохи (различие станет понятно из последующего изложения) или шаги в большинстве случаев будем обозначать через надстрочные или подстрочные $k$, соответственно последующий шаг через $k+1$, а предыдущий $k-1$.

## Методы оптимизации

Сделаем несколько общих замечаний о методах оптимизации. Вспомним, что методы оптимизации могут быть **аналитическими** (например, метод наименьших квадратов) и **численными** (градиентный спуск).

Если численный метод предполагает задание некоторого приближенного решения (*начальное*

*приближение*, $\theta_0$), которое затем улучшается на каждой итерации, то такой метод называется **итерационным** (iterative).

Метод **сходится** (converges) к оптимальному значению, если

$$\theta^k \xrightarrow[k \to \infty]{} \theta^*$$

Также заметим, что методы или алгоритмы оптимизации часто называют **оптимизаторами** (optimizers, solvers) [функции потерь]. Например, упомянутые на вводном курсе метод стохастического градиентного спуска или алгоритм Adam как раз являются примерами оптимизаторов.

Перейдем к теме занятия.

# Подготовка данных

Откроем блокнот к этому занятию

В большинстве примеров будем использовать уже знакомую модель множественной линейной регрессии, а также искусственно созданный (синтетический, synthetic) набор данных.

Напишем функцию, которая будет создавать такие данные. На вход она будет принимать вектор весов theta (включая сдвиг, $\theta_0$), количество наблюдений n, а также «уровень шума» noise.

```
def synth_reg_data(theta, n = 1000, noise = 0.01, random_state = 42):
  np.random.seed(random_state)
  # создадим массив данных, следующий нормальному распределению,
  # размерностью: кол-во наблюдений x кол-во признаков без смещения
  X = np.random.normal(size = (n, len(theta)-1))
  # добавим столбец из единиц для смещения
  X = np.c_[np.ones((len(X), 1)), X]
  # создадим нормально распределенный шум
  noise = np.random.normal(size = n) * noise
  # найдем значения целевой переменной
  y = np.dot(X, theta) + noise
  return X, y
```

Такой подход позволит нам заранее знать оптимальные веса $\theta^*$. Как следствие, мы сможем оценить, как изменяется расстояние от текущих весов до истинных.

```
# создадим вектор весов (первое значение - сдвиг)
t_true = np.array([3.2, 2, -2.7, 1.6])
# сгенерируем данные
X, y = synth_reg_data(t_true)
```

Пропишем функцию градиента модели линейной регрессии.

```
def gradient(x, y, thetas, n):
  return np.dot(x.T, (np.dot(x, thetas) - y)) * (1 / n)
```

# Работа с наблюдениями

Начнем улучшать алгоритм градиентного спуска, обучая модель на различном количестве имеющихся наблюдений.

## Full batch

Вначале объявим функцию линейной регрессии, которая будет обновлять веса на основе *всех* наблюдений. Такой подход называется **full batch** градиентным спуском.

На вход функция fit_full_batch() будет принимать:

- массив признаков x и целевую переменную y;
- истинные веса t_opt;
- количество итераций алгоритма iter; и
- коэффициент скорости обучения lr.

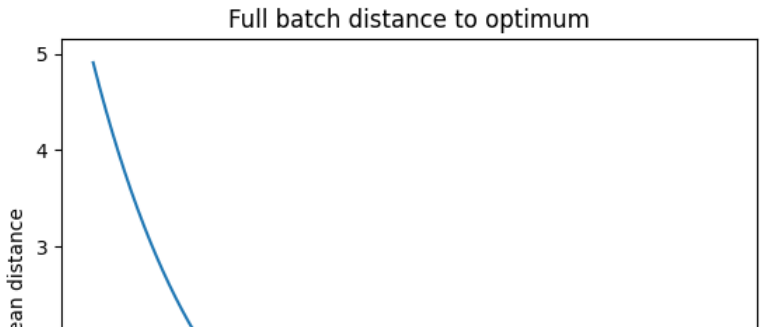
```
def fit_full_batch(x, y, t_opt, iter = 100000, lr = 0.01):

  x, y = x.copy(), y.copy()
  # создадим вектор весов с нулевыми значениями и
  # узнаем количество наблюдений
  thetas, n = np.zeros(x.shape[1]), x.shape[0]
  # создадим список для учета расстояния до оптимальных весов
  dist_to_opt = []

  # в процессе обучения
  for i in range(iter):
    # будем рассчитывать евклидово расстояние до оптимума
    dist_to_opt.append(np.linalg.norm(thetas - t_opt))
    # рассчитаем градиент
    grad = gradient(x, y, thetas, n)
    # и обновим веса
    thetas = thetas - lr * grad

  return dist_to_opt
```

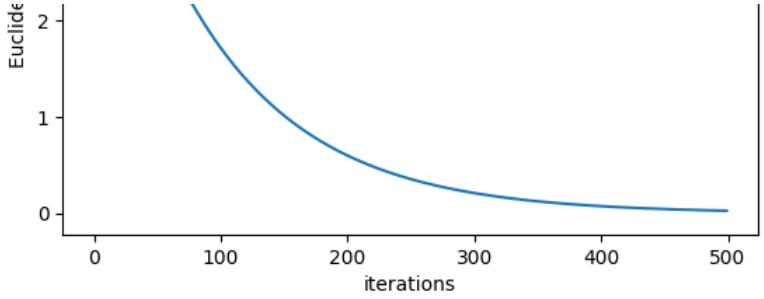
Еще раз заметим, что с помощью цикла мы обновляем веса iter раз, каждый раз рассчитывая градиент на *всех* данных. Обучим модель.

```
dist_fb = fit_full_batch(X, y, t_true)
```

Посмотрим на изменение расстояния до оптимума.

```
plt.plot(dist_fb[:500])
plt.title('Full batch distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance');
```






Как и должно быть, ошибка постоянно и плавно снижается, однако время выполнения кода даже на небольшом наборе данных достаточно существенное.

Математически этот подход можно описать через задачу минимизации относительно $\theta$ конечного множества гладких (непрерывно дифференцируемых) функций потерь

$$\min_{\theta} f(\theta) = \min_{\theta} \frac{1}{n} \sum_{i=1}^{n} f_i(\theta),$$

где каждая $f_i(\theta)$ определяет ошибку для конкретного наблюдения $x_i$. Для линейной регрессии речь идет о

$$f_i(\theta) = (y_i - \theta x_i)^2$$

Усредненная сумма этих ошибок и будет показывать уровень ошибки при текущих весах на имеющихся данных. В таком случае градиентный спуск в общем виде можно определить так

$$\theta^{k+1} := \theta^k - \eta \cdot \nabla f(\theta^k)$$

или так

$$\theta^{k+1} := \theta^k - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} \nabla f_i(\theta^k)$$

для шагов $k = 1, 2, 3, \ldots$

Сложность full batch подхода опять же заключается в том, что расчет градиента на всех наблюдениях вычислительно затратен.

# SGD и Mini batch

## Стохастический градиентный спуск

Попробуем решить задачу минимизации не на всех, а только на одном случайно выбранном наблюдении. В этом случае говорят о **стохастическом** градиентном спуске (stochastic gradient descent или SGD).

Более формально,

$$\theta^{k+1} := \theta^k - \eta \cdot \nabla f_{i_k}(\theta^k), \quad k = 1, 2, 3, \ldots,$$

и $i_k \in \{1, \ldots, n\}$ — случайно выбранный индекс наблюдения на шаге $k$.

Такой алгоритм будет в $n$ раз быстрее, чем обычный градиентный спуск, однако возникает вопрос, будет ли он стремиться к оптимальным параметрам $\theta^*$.

## SGD как несмещенная оценка градиента

Интересно, что хотя спуск в точку минимума очевидно не будет таким плавным, как при full batch подходе (ведь каждый раз мы вычисляем градиент на разных данных), тем не менее на каждом шаге $k$ матожидание стохастического градиентного спуска (для одного наблюдения) $\mathbb{E}[\nabla f_{i_k}(\theta)]$ является несмещенной оценкой (unbiased estimate) full batch градиента $\nabla f(\theta)$.

$$\mathbb{E}[\nabla f_{i_k}(\theta)] = \nabla f(\theta)$$

Разберем это утверждение более подробно. Начнем с того, что несмещенной оценкой называется такая точечная оценка (т.е. число, а не диапазон) некоторого параметра $\theta$, матожидание которого равно этому параметру

$$\mathbb{E}[\hat{\theta}] = \theta, \quad \forall \theta \in \Theta$$

или, что то же самое, $\mathbb{E}[\hat{\theta}] - \theta = 0$. Если же эта разность не равна нулю, то такая величина называется смещением (bias).

$$\mathbb{E}[\hat{\theta} - \theta] = \mathbb{E}[\hat{\theta}] - \mathbb{E}[\theta] = \mathbb{E}[\hat{\theta}] - \theta$$

Далее, по определению математическое ожидание есть средневзвешенная по вероятностям $p_i$ возможных значений $x_i$ оценка случайной величины

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} p_i x_i$$

В случае стохастического градиентного спуска с учетом одинаковой $\frac{1}{n}$ вероятности каждого наблюдения матожидание равно

$$\mathbb{E}[\nabla f_{i_k}(\theta)] = \sum_{i=1}^{n} \frac{1}{n} \nabla f_i(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i(\theta) = \nabla f(\theta)$$

## Mini batch подход

Компромиссом между плавностью спуска и затратами вычислительных ресурсов является использование лишь части (batch) наблюдений или **mini batch** подход. При таком подходе случайным образом выбирается подмножество $I_k \subseteq \{1, \ldots, n\}$ мощностью $|I_k| = b << n$.

$$\theta^{k+1} := \theta^k - \eta \cdot \frac{1}{b} \sum_{i_k \in I_k} \nabla f_{i_k}(\theta^k), \quad k = 1, 2, 3, \ldots$$

Поскольку $\mathbb{E}[\frac{1}{b} \sum_{i \in I_k} \nabla f_{i_k}(\theta)] = \nabla f(\theta)$, матожидание mini batch подхода также будет несмещенной оценкой full batch градиента.

Реализуем эти алгоритмы в коде.

## Деление данных на batches

Начнем с функции, которая будет делить наблюдения на batches.

```
def create_batches(X, y, batch_size, random_state = 42):

  # создадим список, в который будем помещать batches
  mini_batches = []

  # присоединим целевую переменную "справа" от признаков
  data = np.hstack((X, y.reshape((-1, 1))))

  # перемешаем данные случайным образом
  np.random.seed(random_state)
  np.random.shuffle(data)

  # определим количество полных batches
  n_minibatches = data.shape[0] // batch_size

  # если остается неполная batch,
  # увеличиваем общее количество на 1
  if data.shape[0] % batch_size != 0:
    n_minibatches += 1

  # в цикле, равном количеству batches,
  for i in range(n_minibatches):
    # делим наблюдения на mini batches
    mini_batch = data[i * batch_size : (i + 1) * batch_size, :]
    # разделяем признаки и
    X_mini = mini_batch[:, :-1]
    # целевую переменную
    y_mini = mini_batch[:, -1]
    # добавляем эти части в список mini_batches
    mini_batches.append((X_mini, y_mini))

  return mini_batches
```

Протестируем функцию. Разделим данные на batches по 16 наблюдений в каждой.

```
mb_16 = create_batches(X, y, batch_size = 16)
```

С учетом того, что у нас всего 1000 наблюдений, то должно получиться $1000 \div 16 = 63$ полных и неполных batches, из которых 62 будут содержать по 16 наблюдений, а в последней будет $1000 - (62 \times 16) = 8$.

```
len(mb_16), len(mb_16[-2][0]), len(mb_16[-1][0])
```

```
(63, 16, 8)
```

Если же мы хотим реализовать стохастический градиентный спуск, то batch_size должен быть равен единице.

```
mb = create_batches(X, y, batch_size = 1)
len(mb), len(mb[-2][0]), len(mb[-1][0])
```

```
(1000, 1, 1)
```

## Про особенность реализации в коде

Заметим, что при делении данных на batches мы перемешивали данные случайным образом с помощью функции **np.random.shuffle()**, а затем поочередно брали то количество , которое

определялось параметром batch_size. Другими словами, мы выбирали данные без возвращения.

Такой подход нарушает одинаковую вероятность каждого наблюдения $\frac{1}{n}$, а значит не обеспечивает несмещенности оценки стохастического градиентного спуска.

$$\mathbb{E}[\nabla f_{i_k}(\theta)] \neq \nabla f(\theta)$$

Это связано с тем, что выборка без возращения делает события (вероятность выбрать одно наблюдение) зависимыми. Рассмотрим простой пример.

Пусть перед нами две урны $^{(1)}$ и $^{(2)}$. В каждой 7 белых $(w)$ и 3 черных $(b)$ шара. Из первой урны мы будем доставать шары с возвращением, из второй — без.

Найдем вероятность достать белый шар на первом шаге $(w_1)$. Для обеих урн вероятность будет одинаковой.

$$P^{(1)}(w_1) = \frac{7}{10} \qquad P^{(2)}(w_1) = \frac{7}{10}$$

Предположим, что мы действительно достали белый шар. Какой будет вероятность достать черный шар на втором шаге $b_2$? По правилу совместной вероятности независимых событий для первой урны получим

$$P^{(1)}(w_1, b_2) = P^{(1)}(w_1) \cdot P^{(1)}(b_2) = \frac{7}{10} \cdot \frac{3}{10} = 0{,}21$$

В случае второй урны события зависимы. Другими словами, вероятность достать черный шар на втором шаге зависит от первого события, то есть белого шара, $P^{(2)}(b_2 \mid w_1)$, и такая вероятность равна $\frac{3}{9}$ (в урне по-прежнему три черных шара, но всего шаров теперь только девять).

Совместная вероятность зависимых событий, таким образом, будет равна

$$P^{(2)}(w_1, b_2) = P^{(2)}(w_1) \cdot P^{(2)}(b_2 \mid w_1) = \frac{7}{10} \cdot \frac{3}{9} \approx 0{,}23$$

Говоря о реализации SGD в коде, хотя подход с перемешиванием (выборки без возвращения) и нарушает математическое обоснование этого алгоритма, тем не менее применяется на практике, поскольку требует меньших ресурсов, чем случайный выбор одного элемента $i_k$ или подмножества $I_k$ из $n$ наблюдений на каждом шаге $k$.

## Сравнение SGD и mini batch

Обучим модель методом стохастического градиентного спуска и с использованием mini batch. Основное отличие от full batch подхода — у нас появляется два цикла:

- эпохи (epochs), за каждую из которой алгоритм должен «просмотреть» все данные;
- batches, после оценки которой стохастический градиентный спуск обновляет веса.

```
def fit_sgd(x, y, t_opt, batch_size, epochs = 50, lr = 0.01):

  x, y = x.copy(), y.copy()
```

```
    thetas = np.zeros(x.shape[1])

    dist_to_opt = []

    for epoch in range(epochs):
      batches = create_batches(x, y, batch_size)
      for x_b, y_b in batches:

        n = x_b.shape[0]
        dist_to_opt.append(np.linalg.norm(thetas - t_opt))

        grad = gradient(x_b, y_b, thetas, n)
        thetas = thetas - lr * grad

    return dist_to_opt
```

```
dist_sgd = fit_sgd(X, y, t_true, batch_size = 1, epochs = 100)
```

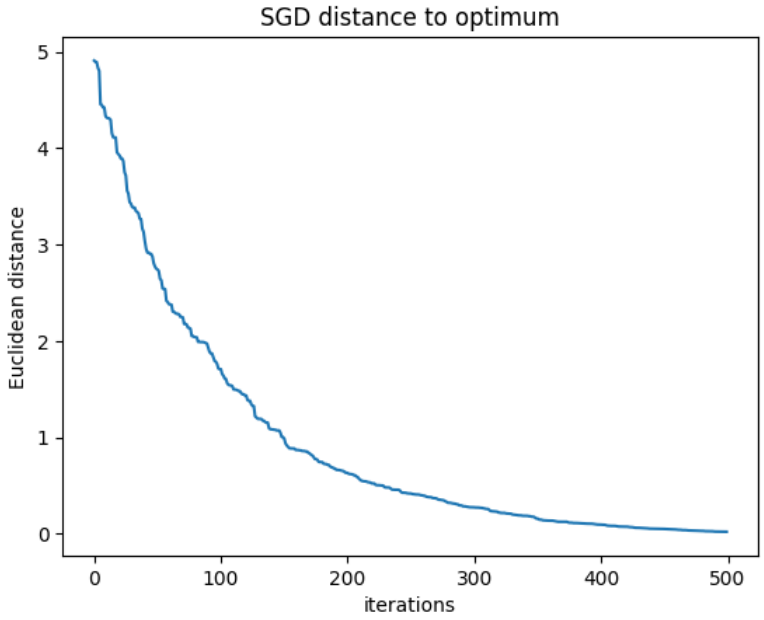
Если количество эпох равно 100, а количество batches 1000 (по одному наблюдению в каждой), то всего алгоритм обновит веса 100000 раз.

```
len(dist_sgd)
```

```
100000
```

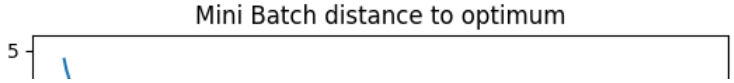
Посмотрим, как менялось расстояние до оптимума.

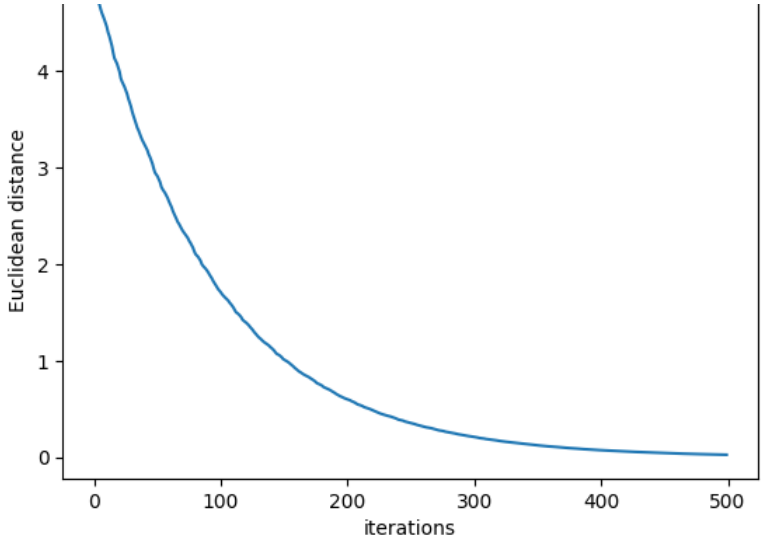
```
plt.plot(dist_sgd[:500])
plt.title('SGD distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance');
```




Как мы видим, присутствуют скачки градиента (и как следствие, расстояния до оптимума). Попробуем увеличить размер партии до 32 наблюдений.

```
plt.plot(fit_sgd(X, y, t_true, batch_size = 32, epochs = 3125)[:500])
plt.title('Mini Batch distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance');
```






Колебания уменьшились, но не исчезли. Возникает вопрос, можно ли сохранить быстроту стохастического спуска и при этом уменьшить дисперсию.

## SVRG

**SVRG** (Stochastic Variance Reduced Gradient) или метод градиентного спуска с уменьшенной дисперсией так же, как и обычный стохастический градиентный спуск рассчитывает градиент для одного наблюдения $\nabla f_{i_k}(\theta)$.

Особенность заключается в том, что шаги разбиты на два вложенных цикла: внешний (outer) и внутренний (inner). В начале каждой итерации внешнего цикла:

- алгоритм запоминает текущие веса $\hat{\theta}$ (theta_hat) и на их основе делает снимок (snapshot) всего градиента $\nabla f(\hat{\theta})$ (g_hat); затем
- случайным образом берет одно наблюдение $x_i$, рассчитывает соответствующий градиент $\nabla f_{i_k}(\theta)$; и
- вычитает из него разницу между градиентом $\nabla f_i(\hat{\theta})$ и общим градиентом $\nabla f(\hat{\theta})$.

$$\theta^{k+1} := \theta^k - \eta \left[ \nabla f_{i_k}(\theta^k) - (f_i(\hat{\theta}) - \nabla f(\hat{\theta})) \right]$$

Таким образом, отличие от SGD состоит в компоненте $(\nabla f_i(\hat{\theta}) - \nabla f(\hat{\theta}))$, который снижает дисперсию стохастического градиента $f_{i_k}(\theta)$ и, хотя и требует дополнительных вычислений, оказывается намного быстрее, чем обычный full batch градиентый спуск.

Более того, поскольку

$$\mathbb{E}[\nabla f_i(\hat{\theta}) - \nabla f(\hat{\theta})] = \mathbb{E}[\nabla f_i(\hat{\theta})] - \mathbb{E}[\nabla f(\hat{\theta})]$$

$$= \nabla f(\hat{\theta}) - \nabla f(\hat{\theta}) = 0$$

то справедливо, что

$$\mathbb{E}\left[ \nabla f_{i_k}(\theta^{(k)}) - (f_i(\hat{\theta}) - \nabla f(\hat{\theta})) \right] = \nabla f(\theta)$$

Другими словами, матожидание SVRG на каждом шаге также является несмещенной оценкой full batch градиента $\nabla f(\theta)$.

Реализуем алгоритм на Питоне.

```
# для svrg потребуется функция расчета градиента,
# не усредняющая градиент на количество наблюдений
def svrg_gradient(x, y, thetas):
  return np.dot(x.T, (h(x, thetas) - y))

def fit_svrg(x, y, t_opt, outer_n = 100, inner_n = 100, lr = 0.01, random_state = 42):

  x, y = x.copy(), y.copy()
  thetas, n = np.zeros(x.shape[1]), x.shape[0]
  dist_to_opt, loss_history = [], []
  np.random.seed(random_state)

  # на каждой итерации внешнего цикла
  for outer in range(outer_n):
    # делаем снимок текущих весов
    theta_hat = thetas

    # и на их основе рассчитываем общий градиент для всех наблюдений
    g_hat = np.zeros(x.shape[1])
    for i in range(n):
      # берем каждое (!) наблюдение
      xi = x[i,:]
      yi = y[i]
      # рассчитываем градиент для этого наблюдения
      # суммируем все градиенты и получаем общий градиент
      g_hat = g_hat + svrg_gradient(xi, yi, theta_hat)
    # усредняем на количество наблюдений
    g_hat = g_hat / n

    # на каждой итерации внутреннего цикла
    for inner in range(inner_n):
      # берем любое (!) наблюдение с одинаковой вероятностью
      i = np.random.randint(n)
      xi = x[i,:]
      yi = y[i]

      dist_to_opt.append(np.linalg.norm(thetas - t_opt))
      loss_history.append(objective(x, y, thetas, n))

      # рассчитываем градиент и
      grad = (svrg_gradient(xi, yi, thetas) - (svrg_gradient(xi, yi, theta_hat) - g_ha
      # обновляем веса
      thetas = thetas - lr * grad

  return dist_to_opt, loss_history
```

Пойдем дальше.

# Инерция

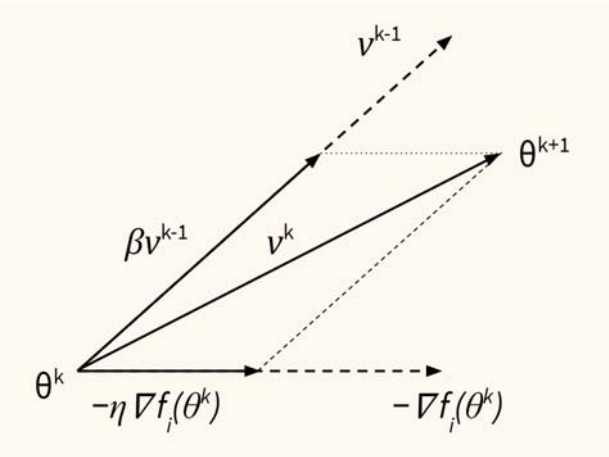
Помимо обучения алгоритма на варьирующемся количестве имеющихся наблюдений, снизить дисперсию можно через использование не только текущего, но и прошлых значений градиента.

# SGD с инерцией

## Физический контекст

Если представить, что вектор весов — это шарик, который скатывается по склону функции потерь, то в каждый последующий момент времени $k+1$ на него будет действовать не только направление градиента на шаге $k$, как в классическом SGD, но и градиент предыдущих шагов, называемый инерцией (momentum, $m^{k-1}$) или вектором скорости (velocity, $v^{k-1}$).

Отсюда и название алгоритма **SGD с инерцией** (with momentum) или SGDm.




Другими словами, веса обновляются под воздействием вектора скорости $v^{k-1}$, скорректированного на величину текущего градиента $\nabla f_i(\theta^k)$.

$$v^k = \beta v^{k-1} + (-\eta \nabla f_i(\theta^k))$$

$$\theta^{k+1} := \theta^k + v^k,$$

где $\beta$ — гиперпараметр, определяющий значимость предыдущих градиентов. И действительно, на графике выше видно, что вектор $v^{k+1}$ является суммой векторов $\beta v^k$ и $-\eta \nabla f_i(\theta^k)$.

Примечание. В этом разделе мы упростили нотацию $\nabla f_{i_k}(\theta^k) \Rightarrow \nabla f_i(\theta^k)$.

Этот же процесс можно записать и так

$$\theta^{k+1} := \theta^k - \eta \nabla f_i(\theta^k) + \beta \underbrace{(\theta^k - \theta^{k-1})}_{v^{k-1}}$$

## Экспоненциальное сглаживание

Одновременно SGDm можно объяснить через экспоненциальное сглаживание (экспоненциально взвешенное скользящее среднее, EWMA), которое используется, в частности, в анализе временных рядов. Рассмотрим следующую рекурсивную функцию

$$v_t = \beta \underbrace{v_{t-1}}_{\text{trend}} + (1-\beta) \underbrace{\theta_t}_{\text{current}},$$

где параметр $\beta$ распределяет значимость между предыдущими $v_{t-1}$ (trend) и текущим $\theta_t$ (current) наблюдением. При этом, чем больше $\beta$, тем большее значение отдается предыдущим наблюдениям.

Формулу SGDm также записывают следующим образом.

$$v^k = \beta v^{k-1} + (1-\beta)\nabla f_i(\theta^k)$$

$$\theta^{k+1} := \theta^k - \eta v^k$$

В такой нотации становится очевидным, что $v^k$ является экспоненциально взвешенным скользящим средним между текущим значением $\nabla f_i(\theta^k)$ и прошлыми градиентами $v^{k-1}$.

Рассмотрим вопрос выбора $\beta$.

## Выбор $\beta$

При $\beta = 0$ речь очевидным образом идет об обычном стохастическом градиентном спуске. Для того чтобы оценить влияние предыдущих градиентов при $\beta > 0$, выпишем первые несколько членов приведенной выше рекурсивной формулы временного ряда.

В качестве начального значения положим $v_0 = 0$.

$$\begin{aligned}
v_0 &= 0 \\
v_1 &= \beta v_0 + (1-\beta)\theta_1 = (1-\beta)\theta_1 \\
v_2 &= \beta v_1 + (1-\beta)\theta_2 = \beta((1-\beta)\theta_1) + (1-\beta)\theta_2 \\
&= (1-\beta)(\theta_2 + \beta\theta_1) \\
v_3 &= \beta v_2 + (1-\beta)\theta_3 \\
&= \beta((1-\beta)(\theta_2 + \beta\theta_1)) + (1-\beta)\theta_3 \\
&= (1-\beta)(\theta_3 + \beta\theta_2 + \beta^2\theta_1)
\end{aligned}$$

Поскольку более ранние наблюдения $\theta$ умножаются на $\beta < 1$, возведенные в возрастающую степень, то их значение или вес со временем уменьшается. Например, на шаге $v_2$ значение первого наблюдения равно $\beta\theta_1$, а на шаге $v_3$ уже $\beta^2\theta_1$.

Приведем общую формулу.

$$v_t = (1-\beta)\sum_{i=0}^{t}\beta^{t-i}\theta_i$$

На практике значение $\beta$ часто выбирается равным 0,9. Изучим, как выбор конкретного $\beta$ влияет на значимость $t$ предыдущих наблюдений $\theta$. Примем без доказательства значение следующего предела.

$$\lim_{x\to 0}(1-x)^{\frac{1}{x}} = \frac{1}{e}$$

Положим $1 - x = \beta$, тогда

$$\lim_{\beta\to 1}\beta^{\frac{1}{1-\beta}} = \frac{1}{e}$$

Заметим, что вес $\beta^t$ определяет значимость слагаемого на шаге $t$. Если допустить $\beta^t = \beta^{\frac{1}{1-\beta}}$, то

$$t = \frac{1}{1-\beta}, \quad \lim_{\beta \to 1} \beta^t = \frac{1}{e}$$

Другими словами, подставляя различные значения $\beta$ в $t = \frac{1}{1-\beta}$, мы можем определить, на каком шаге $t$ вес наблюдения $\beta^t$ приблизится в пределе к $\frac{1}{e} \approx 0{,}368$. Например, при $\beta = 0{,}9$, вес, приблизительно равный $\frac{1}{e}$ будет относиться к $t = \frac{1}{1-\beta} = \frac{1}{1-0{,}9} = 10$-ому шагу.

И действительно,

$$\begin{aligned} \beta^t &= 0{,}9^9 && \approx 0{,}387 && > \tfrac{1}{e} \\ \beta^t &= 0{,}9^{10} && \approx 0{,}349 && \approx \tfrac{1}{e} \\ \beta^t &= 0{,}9^{11} && \approx 0{,}314 && < \tfrac{1}{e} \end{aligned}$$

Если же взять другое значение $\beta = 0{,}95$, то соответствующий вес будет получен на $t = \frac{1}{1-0{,}95} = 20$-ом шаге. Проверим.

$$\begin{aligned} \beta^t &= 0{,}95^{19} && \approx 0{,}377 && > \tfrac{1}{e} \\ \beta^t &= 0{,}95^{20} && \approx 0{,}358 && \approx \tfrac{1}{e} \\ \beta^t &= 0{,}95^{21} && \approx 0{,}341 && < \tfrac{1}{e} \end{aligned}$$

Обратим внимание, при $\beta \to 1$ значение $\beta^t$ действительно приближается к $\frac{1}{e}$. При $\beta = 0{,}99$,

$$t = \frac{1}{1-0{,}99} = 100, \quad \beta^t = 0{,}99^{100} = 0{,}366$$

Для чего нужны эти расчеты? На практике, веса, меньшие, чем $\frac{1}{e}$ не имеют большого значения при расчете текущего значения $\theta$ и соответственно с помощью $t = \frac{1}{1-\beta}$ можно понять, сколько предыдущих $t$ шагов в наибольшей степени влияют на текущее наблюдение.

Приведем расчет количества значимых шагов $t$ для некоторых $\beta$.

| $\beta$ | $\|\|$ | 0,5 | 0,8 | 0,9 | 0,95 | 0,98 | 0,99 |
|---|---|---|---|---|---|---|---|
| $t = \frac{1}{1-\beta}$ | $\|\|$ | 2 | 5 | 10 | 20 | 50 | 100 |

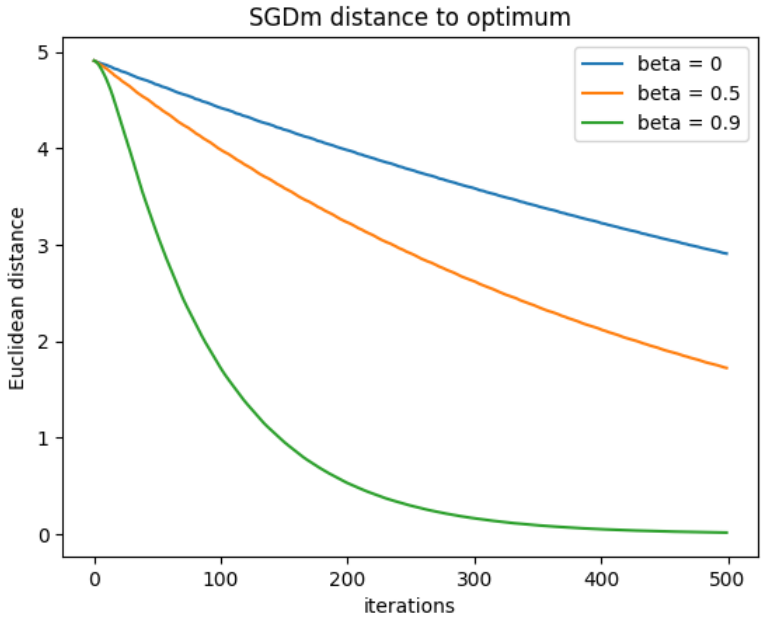
## Реализация на Питоне

Вернемся к стохастическому градиенту с инерцией и реализуем алгоритм SGDm на Питоне. Одновременно посмотрим, как различные значения $\beta$ влияют на сходимость и уровень колебаний.

```
def fit_sgdm(x, y,
             t_opt,
             batch_size = 32,
             epochs = 3125,
             lr = 0.001,
             momentum = 0.9):

  x, y = x.copy(), y.copy()

```

```
  # объявляем вектор весов и
  thetas = np.zeros(x.shape[1])
  # вектор инерции
  v = np.zeros(x.shape[1])

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      v = momentum * v - lr * gradient(x_b, y_b, thetas, n)
      thetas += v

  return dist_to_opt
```

```
plt.plot(fit_sgdm(X, y, t_true, momentum = 0.0)[:500], label = 'beta = 0')
plt.plot(fit_sgdm(X, y, t_true, momentum = 0.5)[:500], label = 'beta = 0.5')
plt.plot(fit_sgdm(X, y, t_true, momentum = 0.9)[:500], label = 'beta = 0.9')
plt.legend()
plt.title('SGDm distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance');
plt.show()
```




Примечание. В коде выше вес прошлых градиентов $\beta$ обозначен как momentum, хотя инерцией или трендом, как мы сказали, называется компонент $v^{k-1}$. Такое название переменной выбрано для того, чтобы сохранить единообразие с основными библиотеками для обучения нейросетей, например, с Keras.

## Сглаживание без смещения

Прогнозирование тренда (информации о предыдущих шагах в случае SGDm) с использованием экспоненциально взвешенного скользящего среднего имеет один недостаток. Рассмотрим следующий временной ряд.

```
# 30 наблюдений
obs = np.array([33, 24, 31, 37, 36, 28, 31, 29, 35, 37, 38, 39, 36, 33, 34, 32, 35, 35,
```

Если положить $v_0 = 0$, то в соответствии с формулой экспоненциального сглаживания в первые несколько шагов значительный вес будет отдан именно нулевому значению. При этом значения первого и последующих наблюдений могут быть далеки от нуля, и скользящее среднее этого не учтет.

$$\begin{aligned} v_0 &= 0 \\ v_1 &= 0{,}9 \cdot 0 + (1 - 0{,}9) \cdot 33 = 3{,}3 \\ v_2 &= 0{,}9 \cdot 3{,}3 + (1 - 0{,}9) \cdot 24 = 5{,}37 \end{aligned}$$

Возникает заметное **смещение** (bias). Для того чтобы справиться с этим, мы могли бы положить $v_0 = \theta_1$. Однако это решает проблему лишь частично. Если, например, второе и последующие наблюдения $\theta_2, \theta_3, \ldots$ отличаются от первого, то опять же скользящее среднее будет в большей степени зависеть именно от $\theta_1$.

$$\begin{aligned} v_0 &= 33 \\ v_1 &= 0{,}9 \cdot 33 + (1 - 0{,}9) \cdot 33 = 33 \\ v_2 &= 0{,}9 \cdot 33 + (1 - 0{,}9) \cdot 24 = 32{,}1 \end{aligned}$$

Более удачным представляется деление скользящего среднего на $1 - \beta^t$.

$$v_t = \frac{\beta v_{t-1} + (1 - \beta)\theta_t}{1 - \beta^t}$$

В этом случае, так как $1 - \beta^t$ меньше единицы, мы увеличиваем скользящее среднее. Одновременно, с ростом $t$ значение $1 - \beta^t$ стремится к одному, и его эффект на $v_t$ становится меньше. Ровно то, что нам нужно.

Вычислим первые несколько значений.

$$\begin{aligned} v_0 &= 0 \\ v_1 &= \frac{0{,}9 \cdot 0 + (1 - 0{,}9) \cdot 33}{1 - 0{,}9^1} = 33 \\ v_2 &= \frac{0{,}9 \cdot 3{,}3 + (1 - 0{,}9) \cdot 24}{1 - 0{,}9^2} \approx 28{,}26 \end{aligned}$$

Обратим внимание на важный нюанс. На очередном шаге в формулу скользящего среднего $v_t$ мы подставляем предыдущее $v_{t-1}$ значение экспоненциального сглаживания *без учета корректировки* $1 - \beta^t$ (сравните значения выделенные цветом).

Остальные расчеты проведем на Питоне.

```
beta = 0.9

# создадим два списка для обычного и несмещенного EWMA
# начальное значение v0 = 0
exp_smoothing = [0]
exp_smoothing_unbiased = [0]

# индекс i (в формуле обозначается t) идет от 1 до 30
for i in range(1, len(obs) + 1):
  # индекс сглаживания exp_smoothing[i - 1] и наблюдений obs[i - 1]
  # от 0 до 29, чтобы рассчитать EWMA для наблюдений от 1 до 30
  v = beta * exp_smoothing[i - 1] + (1 - beta) * obs[i - 1]

  # добавим очередное значение EWMA в списки
```

```
    exp_smoothing.append(v)
    exp_smoothing_unbiased.append(v / (1 - beta ** i))
```

И снова обратите внимание, хотя мы добавляем в список `exp_smoothing_unbiased` значение с учетом `(1 - beta ** i)`, значение переменной `v` переходит на следующий шаг без корректировки.

Создадим датафрейм для наблюдений и значений обычного и несмещенного экспоненциально взвешенного скользящего среднего.

```
smoothing = pd.DataFrame({'obs':np.insert(obs, 0, 0),
                          'regular_EWMA':exp_smoothing,
                          'unbiased_EWMA':exp_smoothing_unbiased}).round(2)

smoothing.iloc[0,0] = np.NaN
```

Посмотрим на первые и последние значения.

```
smoothing.head()
```

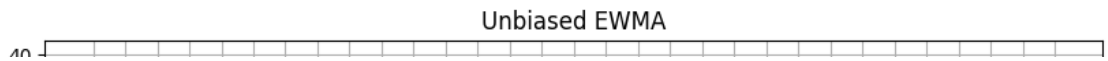
| | obs | biased_EWMA | unbiased_EWMA |
|---|---|---|---|
| **26** | 26.0 | 30.84 | 32.97 |
| **27** | 30.0 | 30.76 | 32.66 |
| **28** | 32.0 | 30.88 | 32.59 |
| **29** | 34.0 | 31.19 | 32.74 |
| **30** | 35.0 | 31.57 | 32.97 |

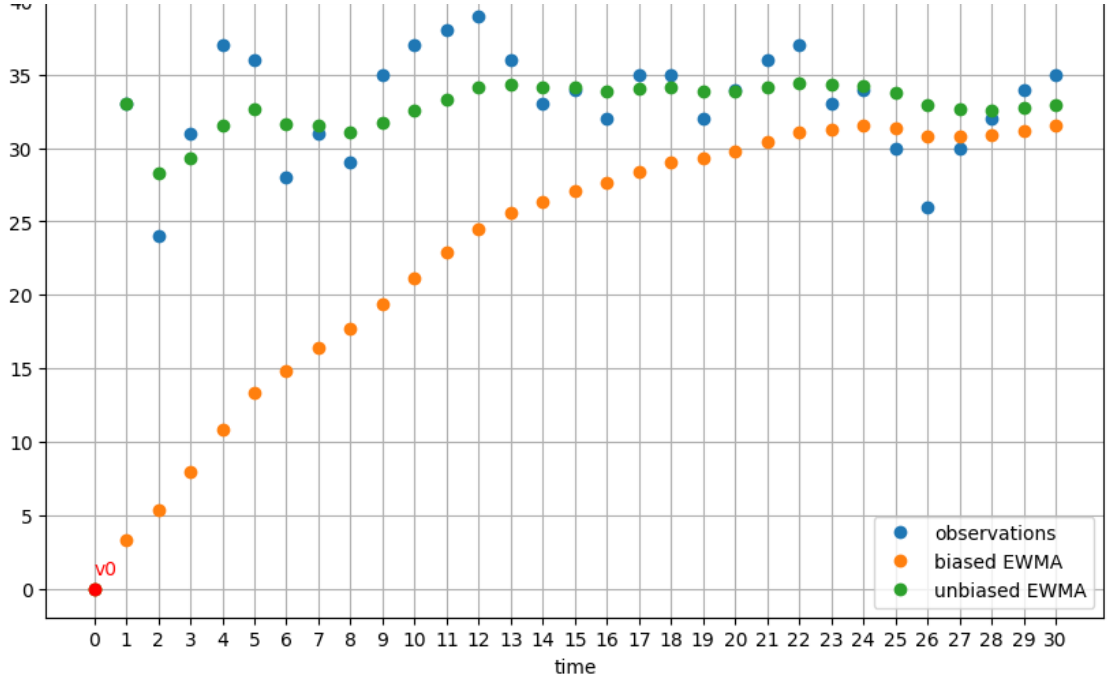
```
smoothing.tail()
```

| | obs | biased_EWMA | unbiased_EWMA |
|---|---|---|---|
| **26** | 26.0 | 30.84 | 32.97 |
| **27** | 30.0 | 30.76 | 32.66 |
| **28** | 32.0 | 30.88 | 32.59 |
| **29** | 34.0 | 31.19 | 32.74 |
| **30** | 35.0 | 31.57 | 32.97 |

Несложно заметить, что эффект корректировки на первых значениях выражен сильнее, чем на последних. Посмотрим на эти данные на графике.

```
time = np.arange(1, 31)

plt.figure(figsize=(10,6))
plt.plot(time, obs, 'o', label = 'observations')
plt.plot(exp_smoothing, 'o', label = 'regular EWMA')
plt.plot(exp_smoothing_unbiased, 'o', label = 'unbiased EWMA')
plt.plot(0, 0, 'ro')
plt.annotate('v0', (0, 1), color = 'r')
plt.legend()
plt.title('Unbiased EWMA')
plt.xlabel('time')
plt.xticks(np.arange(31))
plt.grid()
plt.show()
```






Эту же идею можно использовать и в градиентном спуске.

$$v^k = \beta v^{k-1} + (1-\beta)\nabla f_i(\theta^k)$$

$$\hat{v}^k = \frac{v^k}{1-\beta^k}$$

$$\theta^{k+1} := \theta^k - \eta \hat{v}^k,$$

```
def fit_sgdm_unbiased(x, y,
                      t_opt,
                      batch_size = 32,
                      epochs = 3125,
                      lr = 0.001,
                      momentum = 0.9):

  x, y = x.copy(), y.copy()

  thetas = np.zeros(x.shape[1])
  v = np.zeros(x.shape[1])
  k = 1

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      v = momentum * v + (1 - momentum) * gradient(x_b, y_b, thetas, n)
      v_hat = v / (1 - momentum ** k)
      thetas -= lr * v_hat
      k += 1

  return dist_to_opt
```

Вновь обратим внимание, на следующий шаг переходит значение $v^k$ без корректировки.

### Dampening

Еще раз рассмотрим уже знакомую нотацию SGDm.

$$v^k = \beta v^{k-1} + (1-\beta)\nabla f_i(\theta^k)$$

$$\theta^{k+1} := \theta^k - \eta v^k$$

Если заменить коэффициент $(1-\beta)$ на $(1-\tau)$, где $\tau$ — коэффициент затухания (dampening), мы смогли бы отдельно контролировать значимость градиента.

$$v^k = \beta v^{k-1} + (1-\tau)\nabla f_i(\theta^k)$$

$$\theta^{k+1} := \theta^k - \eta v^k$$

В случае $\tau = \beta$ речь очевидно идет об обычном SGDm.

```
def fit_sgdm_dampening(x, y,
                       t_opt,
                       batch_size = 32,
                       epochs = 3125,
                       lr = 0.001,
                       momentum = 0.9,
                       dampening = 0.1):

  x, y = x.copy(), y.copy()

  thetas = np.zeros(x.shape[1])
  v = np.zeros(x.shape[1])

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      v = momentum * v + (1 - dampening) * gradient(x_b, y_b, thetas, n)
      thetas -= lr * v

  return dist_to_opt
```

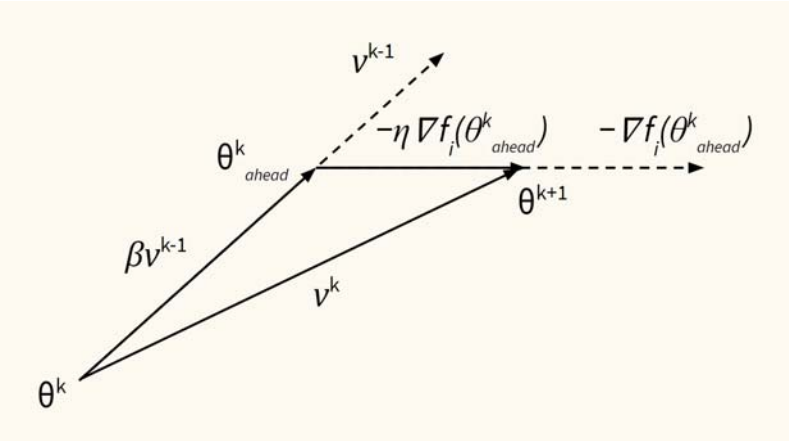
Подобный функционал реализован, в частности, в библиотеке PyTorch.

## Nesterov momentum

Представим, что информация о предыдущих шагах $v^k$ ведёт к неоптимальному изменению параметров $\theta$. Возвращаясь к аналогии с физическим миром, предположим, что место, куда ведёт инерция, для шага $k+1$ вскоре сменится подъемом, и шарик соответственно замедлится.

Если рассчитать изменение текущего градиента именно в точке, куда ведёт прошлый градиент, так называемый, lookahead, $\theta^k_{\text{ahead}}$, то такое потенциально неудачное перемещение можно скорректировать сильнее, чем мы бы это сделали в точке $\theta^k$, пользуясь обычным методом SGDm.

Этот метод, предложенный Ю.Е. Нестеровым, получил название **Nesterov Momentum** или Nesterov Accelerated Gradient (NAG).




Таким образом, для обновления весов необходимо:

1. найти точку (значения весов в точке) $\theta^k_{\text{ahead}}$, на которую указывает прошлый градиент $\beta v^{k-1}$;
2. вычислить текущий градиент $-\eta \nabla f_i(\theta^k_{\text{ahead}})$ и найти новый шаг, т.е. сумму векторов $v^k = \beta v^{k-1} + (-\eta \nabla f_i(\theta^k_{\text{ahead}}))$;
3. обновить веса с $\theta^k$ до $\theta^{k+1}$ с помощью $v^k$.

Более формально,

$$\theta^k_{\text{ahead}} = \theta^k + \beta v^{k-1}$$

$$v^k = \beta v^{k-1} + (-\eta \nabla f_i(\theta^k_{\text{ahead}}))$$

$$\theta^{k+1} := \theta^k + v^k$$

Напишем код на Питоне.

```
def fit_nesterov(x, y,
                 t_opt,
                 batch_size = 32,
                 epochs = 3125,
                 lr = 0.001,
                 momentum = 0.9):

  x, y = x.copy(), y.copy()

  thetas = np.zeros(x.shape[1])
  v = np.zeros(x.shape[1])

  dist_to_opt = []

  v = np.zeros(x.shape[1])

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]
      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      thetas_ahead = thetas + momentum * v
```

```
        v = momentum * v - lr * gradient(x_b, y_b, thetas_ahead, n)
        thetas += v

    return dist_to_opt
```

## Промежуточный итог

В первой части занятия мы посмотрели, каким образом можно улучшить алгоритм градиентного спуска, (1) варьируя количество наблюдений, на основе которых рассчитывается градиент, а также (2) используя информацию о предыдущих шагах.

## Список литературы


- A Stochastic Approximation Method, Robbins and Monro, 1951
- О некоторых способах ускорения сходимости итерационных методов, Б.Т. Поляк, 1964
- Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour, Goyal et al., 2018
- Accelerating Stochastic Gradient Descent using Predictive Variance Reduction, Johnson & Zhang, 2013